

It Is Not the Cryptography: A Discarded Exception Dominates Token Validation Cost in a Node.js Service

Katha Sikdar¹

¹Independent Researcher.

Contributing authors: ferdouskatha35@gmail.com;

Abstract

The per-request cost of JSON Web Token validation is routinely attributed to cryptography. We show that in a widely used Node.js library the attribution is wrong, and that the true mechanism is neither cryptography nor, as we ourselves previously reported, the conversion of key material. When the caller supplies a shared secret as a string — which is what the library’s interface invites — `jsonwebtoken` resolves it by attempting to parse it as an *asymmetric* key first, and falling back to the symmetric constructor only after that attempt throws. The conversion it falls back to costs $0.583\ \mu\text{s}$. The discarded failure that precedes it costs $18.04\ \mu\text{s}$, or 60% of the entire validation call. Measuring across four container runtimes on one machine, we find the magnitude of this cost is governed by the bundled OpenSSL version rather than by containerisation, virtualisation or the JavaScript engine: two runtimes sharing the OpenSSL 3.0 series but differing in V8 major version both pay roughly 373.79 – $420.62\ \mu\text{s}$, while an OpenSSL 3.5 runtime pays $22.94\ \mu\text{s}$. Everything else in the validation path varies by at most $1.26\times$ across the same runtimes; the discarded attempt varies by $25.36\times$. A survey of public code finds the avoiding pattern essentially absent: of 1124352 files importing the library, 130 also mention the pre-parsing API, and in a sample of 315 call sites across 235 repositories, 0 pass a pre-parsed key. We propose a fix, and report a constraint on it that we believe is the more important result: the discarded attempt is load-bearing for an existing key-type defence, so the obvious form of the optimisation must not be applied.

Keywords: JSON Web Token, performance measurement, benchmarking methodology, Node.js, OpenSSL, software defect

1 Introduction

Authentication in cloud-native services is commonly described as imposing a per-request cryptographic cost. The reasoning is intuitive: validating a JSON Web Token means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* — to a sidecar proxy, a gateway, or dedicated hardware — because specialised components do cryptographic work well.

We report that for the most widely deployed JavaScript implementation the premise is wrong, and that our own earlier attempt to correct it was also wrong, in a way worth recording. That earlier work established that the signature primitive is a negligible share of the cost and attributed the remainder to converting key material from a string into a key object on every call. Measuring the conversion directly shows it costs $0.583\ \mu\text{s}$ — too small to be the explanation for anything.

The actual mechanism is a control-flow accident. Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed. Every validation therefore constructs and discards an OpenSSL parse failure, at $18.04\ \mu\text{s}$ against a total call cost of $29.92\ \mu\text{s}$.

Contributions

1. A decomposition, by two independent methods, locating the dominant cost of token validation in a discarded exception rather than in cryptography or in key conversion (Section 3).
2. Evidence that the magnitude of this cost is governed by the runtime’s bundled OpenSSL version rather than by containerisation, virtualisation or the JavaScript engine, using a runtime pair that separates OpenSSL from V8 (Section 4).
3. A survey of public code establishing that the pattern which avoids the cost is close to absent in practice (Section 5).
4. A fix, together with a constraint on it: the discarded attempt is load-bearing for an existing key-type defence, and the natural form of the optimisation is therefore unsafe (Section 6).
5. A correction of our own previously reported mechanism, stated explicitly rather than quietly revised (Section 7).

2 Methodology

2.1 What is measured, and at what granularity

The unit of measurement is one operating-system process, not one call. Each process performs 40000 timed iterations of a single condition after a warmup of half that number, and contributes its median. Reported figures are medians of per-process medians, with percentile bootstrap intervals taken over processes.

This matters because the alternative is misleading in a specific way. An interval computed over the calls inside one process describes that process’s steady state and is far too narrow: it cannot observe the compilation decisions, inline-cache states and heap layouts that differ between runs of the same code, which is precisely what makes a microbenchmark hard to reproduce. Every condition here was run in 15 separate processes.

Between-process coefficient of variation did not exceed 3.5% on any condition.

Conditions are interleaved within each round rather than run in blocks. Under block ordering, a thermal excursion or a background process lands entirely on whichever condition was executing and is indistinguishable from an effect; interleaved, it is distributed across conditions and appears as between-round variance, which the intervals then report.

The clock's own cost is measured in the same process as the condition it accompanies and is reported rather than subtracted: 0.042 μ s against the smallest quantity we report, 0.583 μ s.

2.2 Decomposing the call

`jwt.verify()` is decomposed into the operations it performs, each timed in isolation under the same protocol: structural decode; the HMAC primitive with a string key and with a pre-parsed key; the symmetric key constructor; and the asymmetric key parse that the library attempts first, in both the case where it throws and the case where it succeeds.

2.3 Separating the runtime from the deployment

Our earlier work compared a host micro-benchmark against an in-container measurement and attributed the difference to deployment. Those two environments differed in more than deployment, so the comparison could not support the attribution. Here containerisation is held constant and the runtime is varied: four container images spanning two OpenSSL series and three V8 major versions, all on one machine, under one container engine.

The decisive pair is Node 18 and Node 20. They ship different V8 major versions and share the OpenSSL 3.0 series. In

stock images Node and OpenSSL versions otherwise co-vary, which would leave the attribution underdetermined.

2.4 Independent confirmation

The decomposition above reaches its conclusion by subtracting conditions — the same inference style this paper criticises elsewhere. It is therefore checked against V8's sampling profiler, which attributes self time to named frames with no arithmetic on our part.

2.5 Prevalence

Three measurements with different failure modes. Population counts from code search over public repositories; a sample of call sites classified by tracing the key argument within its file; and a targeted counter-search for the avoiding pattern, fetched in full and adjudicated by hand. The third exists because a sample can only show a pattern is rare if it could have found it.

3 The mechanism

3.1 Where the time goes

Table 1 gives the decomposition on the host.

The two bold rows are the result. The conversion to which the cost has been attributed — including by us — costs 0.583 μ s. The failure that precedes it costs 18.04 μ s, which is 60% of the whole call.

Passing a pre-parsed key instead of a string reduces the call by 23.92 μ s [23.58, 24.00] over 15 paired rounds.

Table 1 Cost of each operation in the validation process. Medians of per-process medians over 15 processes, 40000 calls per each; 95% percentile bootstrap intervals.

Operation	Median (μ s)	95% CI
verify(), string secret	29.92	[29.75, 30.12]
failed asymmetric parse	18.04	[17.83, 18.25]
successful asymmetric parse	13.83	[13.62, 13.92]
verify(), pre-parsed key	6.04	[5.96, 6.12]
HMAC, string key	2.21	[2.21, 2.25]
HMAC, pre-parsed key	2.17	[2.12, 2.17]
structural decode	1.29	[1.25, 1.29]
symmetric key constructor	0.583	[0.54, 0.58]
timer overhead	0.042	[0.04, 0.04]

Table 2 The same measurement across four containerisation on one machine, 10 processes each, 20000 calls per process. The host row is from Table 1.

Operation	Median (μ s)	95% CI	OpenSSL	Failed parse (μ s)	Pre-parsed verify()
verify(), string secret	29.92	[29.75, 30.12]	—	—	—
failed asymmetric parse	18.04	[17.83, 18.25]	3.0.16	373.79	7.58
successful asymmetric parse	13.83	[13.62, 13.92]	3.0.19	420.62	7.25
verify(), pre-parsed key	6.04	[5.96, 6.12]	3.5.7	22.94	6.56
HMAC, string key	2.21	[2.21, 2.25]	3.5.7	16.58	6.04
HMAC, pre-parsed key	2.17	[2.12, 2.17]	—	18.04	6.04
structural decode	1.29	[1.25, 1.29]	—	—	—
symmetric key constructor	0.583	[0.54, 0.58]	—	—	—
timer overhead	0.042	[0.04, 0.04]	—	—	—

3.2 The parts account for most, but not all, of the penalty

If the penalty were exactly the discarded failure plus the conversion, the two would sum to it. They do not, quite:

observed penalty	23.92 μ s
failed parse + conversion	18.62 μ s
residual	5.29 μ s

The residual’s interval, [4.92, 5.50], excludes zero, so the two measured parts are not the whole story: they account for 77.9%, and roughly 5.29 μ s is the library’s own type and claim checking along the non-pre-parsed path. We report this rather than absorb it into the headline.

3.3 Confirmation without differencing

V8’s sampling profiler attributes 67.9% of self time on the host to the asymmetric key parse when a string secret is supplied. Under the Node 18 runtime the same frame accounts for 94.7%; under Node 26, 57.9%. The frame does not appear among the top frames of the pre-parsed path at all. Two methods, one answer.

4 The magnitude is a property of the runtime

Table 2 varies the runtime with containerisation held constant.

Three things follow.

It is not containerisation.

Containerised Node 26 on glibc costs 16.58 μ s against 18.04 μ s on the uncontainerised host, a ratio of 0.92. The container is marginally *faster*. Whatever produces the large costs in this table, the act of running inside a container is not it.

It is OpenSSL, not the JavaScript engine.

Node 18 and Node 20 ship different V8 major versions and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μ s. Node 26 ships OpenSSL 3.5 and pays 22.94 μ s on the same libc. Because Node and OpenSSL versions covary in stock images, this attribution is normally underdetermined; the Node 20 runtime is what separates them.

The effect is confined to key parsing.

Across every environment measured, pre-parsed validation spans 1.26 $\times$, the HMAC primitive 1.47 $\times$ and structural decode 1.69 $\times$. The failed parse spans

25.36 $\times$. The successful parse moves with it (11.74 $\times$), so this is the cost of OpenSSL 3.0 key parsing in general rather than anything peculiar to the failure path. The failure is simply the case in which an application pays that cost on every request for a result it discards.

5 How common is the pattern?

A defect that nobody triggers is a curiosity. Three measurements.

Population counts.

Of 790528 public JavaScript files importing the library, 29 also mention the pre-parsing API; for TypeScript the figures are 333824 and 101. Together 130 of 1124352, or 0.012% — about one file in 8649.

Sampled call sites.

315 call sites across 235 distinct repositories, one file per repository so that no single project dominates. Environment variables account for 176, untraced identifiers and member expressions 85, string literals 49, file contents 3 and buffers 2. Pre-parsed keys account for 0. No file in this corpus contains the pre-parsing API at all, so the zero reflects the pattern’s absence rather than a classifier that cannot see it.

Incidentally, 299 of 315 call sites (95%) pass no algorithm restriction. This is not what the survey set out to measure, but it bears directly on Section 6.

Counter-search, adjudicated by hand.

Every file co-occurring the library with the pre-parsing API was fetched: 191 files, of which 120 contain a validation call.

An automated pass flagged 13 as passing a pre-parsed key; all were then read. Of these, 5 are false positives — projects defining their own helper function of the same name, one of which returns a string — and 1 could not be resolved within the file. Genuine: 7 call sites across 6 projects.

6 A fix, and a constraint on it

The fix is to dispatch on the token’s declared algorithm before attempting the asymmetric parse, so that an HMAC token with an ordinary string secret is resolved directly. Applied, this takes the call from 29.92 μ s to 7.21 μ s, a factor of 4.15 and within 1.21 μ s of supplying a pre-parsed key, with no change in calling code.

The constraint is the more important half. The discarded attempt is not purely wasteful: the *success* of that parse on asymmetric material is what produces a key object whose type a later check depends on, and that check is what prevents symmetric and asymmetric key material from being interchanged. A change that dispatches on the declared algorithm while relaxing which key material may take the fast path alters which inputs reach that check.

The fix must therefore be narrow: the fast path is restricted to string secrets that cannot be asymmetric key material, so that anything which might be resolves exactly as it does today. We verified that the narrow form preserves the check and that a form without the material restriction does not. Two observations sharpen the point. First, a regression test that passes on both a correct and an incorrect build constrains nothing; the test reported here fails on the unrestricted form, which is what makes it evidence

rather than decoration. Second, the independent defence against this class of confusion is for the caller to restrict permitted algorithms — and 95% of surveyed call sites do not.

7 Correcting our own earlier report

Our earlier manuscript reported that the dominant term was key material converted from a string on every call. That is wrong: the conversion costs $0.583\ \mu\text{s}$. It also explained an asymmetry between HMAC and RSA costs as conversion being dearer than parsing a PEM, which is backwards — a successful asymmetric parse costs $13.83\ \mu\text{s}$ against $0.583\ \mu\text{s}$ for the conversion. The real asymmetry is that the RSA path’s parse *succeeds*, making its cost useful work, while the HMAC path’s fails, making its cost waste.

That manuscript further compared a host microbenchmark against an in-container measurement and attributed the difference to deployment. Section 4 shows the comparison spanned two OpenSSL major series, so it could not support that attribution. It also stated a runtime version for the system under test that was read from the host rather than from the container, and was wrong by two major versions.

We record these rather than revise them silently because the errors share a structure worth naming: in each case a quantity was attributed to the variable we were interested in, when a second variable had changed at the same time and had not been recorded at the granularity that would have revealed it.

8 Threats to validity

One machine, one instruction set.

Every measurement in this paper comes from a single ARM64 host: an Apple silicon laptop, its container runtimes under one engine. We have **not** verified any of this on x86_64.

We state what we expect and why, and then state that we did not test it. The mechanism is a property of a library’s API contract — an asymmetric parse attempted before a symmetric one, with the failure discarded — and of the cost of key parsing in a particular OpenSSL series. Neither is a property of an instruction set, and we know of no reason for the control flow to differ on x86_64. But an expectation is not a measurement. The magnitude in particular depends on how OpenSSL’s key parsing performs on a given platform, and that is exactly the kind of quantity that does vary with architecture. Until the following is filled in, no claim in this paper should be read as holding beyond ARM64: **[MISSING: xEightSixReplication]**.

One library, one version.

The mechanism is specific to one JavaScript implementation at one version. Whether other implementations or other language ecosystems attempt key resolution in the same order is unmeasured.

The prevalence survey measures files, not systems.

A project that constructs a pre-parsed key in one file and validates in another is invisible to file-level analysis, and this is the strongest threat to Section 5. Within-file tracing found no such case, which is weak evidence against it. Code search covers public repositories only and includes

forks, vendored dependencies and teaching material; the numerator is inflated in the same way as the denominator, but the ratio is not a clean population proportion and should not be read as one. Public code also skews toward small projects, and the 6 projects found using the API correctly are all small.

Absolute figures are from a shared laptop.

The host also ran a desktop environment. Contention inflates a duration rather than deflating it, so the reported costs are upper bounds and the comparisons between conditions — which is where every claim here rests — are unaffected in direction. Absolute values should not be compared against dedicated hardware.

The host’s cryptographic library moved mid-study.

Installing an unrelated command-line tool upgraded the host’s OpenSSL beneath an unchanged Node binary, because the runtime links it dynamically. No reported measurement is affected: the affected runs predate the upgrade and recorded their OpenSSL version, and container measurements bundle their own. It was detectable only because that version is recorded per measurement rather than per run, which we note as a methodological recommendation rather than an incident.

9 Conclusion

The per-request cost of token validation in this library is real, is worth attending to, and is not cryptographic. It is not key conversion either. It is a parse attempted in the wrong order, whose failure is constructed and thrown away on every request, and whose price is set by

the OpenSSL version a base image happens to pin — varying $25.36\times$ across runtimes while the rest of the validation path varies by at most $1.26\times$. The pattern that avoids it is close to absent from public code.

The methodological lesson is not that one should measure in deployment. It is narrower and, we think, more useful: attribute a cost only to a variable you recorded at the granularity at which it can change. Both errors we correct here — our own, and the field’s — have that shape.

Declarations

Data and code availability. All measurements, the instrumentation, the analysis and the scripts that produce every numeric value in this paper are in the replication package. Every number is generated from a file; none is entered by hand. Quantities that were not measured appear as red placeholders rather than as absences.

Competing interests. The author declares no competing interests.